

A6 Functions & Modularity - Answers

Python - Modular Programming & Functions

MCQs

1. Which keyword is used to define a function in Python?

- ☐ A) function
- ☐ B) define
- ☐ C) def
- ☐ D) fun

Answer: C) def is the Python keyword used to define a function.

2. What is the main purpose of modular programming?

- ☐ A) To make programs longer
- ☐ B) To divide a program into smaller, reusable components
- ☐ C) To avoid using variables
- ☐ D) To eliminate functions

Answer: B) Modular programming divides a program into smaller reusable components.

3. Which of the following correctly defines a function?

- ☐ A) function add(a,b):
- ☐ B) def add(a,b):
- ☐ C) define add(a,b):
- ☐ D) func add(a,b):

Answer: B) def add(a, b): is valid Python function syntax.

4. What is the output of the following code?

```
def square(x):  
    return x * x  
print(square(5))
```

- A) 10
- B) 20
- C) 25
- D) 125

Answer: C) square(5) returns $5 * 5 = 25$.

5. What is the purpose of the return statement in a function?

- ☐ A) To print the function
- ☐ B) To terminate Python
- ☐ C) To send a value back to the caller
- ☐ D) To define a parameter

Answer: C) return sends a result back to the caller.

6. What is the output?

```
def add(a, b):  
    return a + b  
x = add(10, 20)  
print(x)
```

- A) 10
- B) 20
- C) 30
- D) None

Answer: C) add(10, 20) returns and prints 30.

7. Which of the following is a parameter ?

```
def multiply(x, y):  
    return x * y
```

- A) multiply
- B) x and y
- C) return
- D) x * y

Answer: B) x and y are parameters in the function definition.

8. In the following statement, what are 10 and 20?

```
add(10, 20)
```

- A) Parameters
- B) Arguments
- C) Functions
- D) Variables only

Answer: B) 10 and 20 are arguments passed to the function call.

9. What happens if a function does not contain a return statement?

- o A) It always returns 0
- o B) It returns None
- o C) It produces a syntax error
- o D) It returns False

Answer: B) A function without return returns None by default.

10. What is the output?

```
def test():  
    x = 10  
    test()  
print(x)
```

- A) 10

- B) None
- C) NameError
- D) 0

Answer: C) x is local to test(), so print(x) raises NameError.

11. The variable x defined inside a function is generally:

- A) A global variable
- B) A local variable
- C) A class variable
- D) A constant

Answer: B) A variable defined inside a function is generally local.

12. Which is an advantage of using functions?

- A) Code reuse
- B) Better organization
- C) Easier testing and debugging
- D) All of the above

Answer: D) Functions support reuse, organization, testing, and debugging.

13. What is the output?

```
def greet(name="Student"):
    return "Hello " + name
print(greet())
```

- A) Hello
- B) Hello Student
- C) Student
- D) Error

Answer: B) The default argument produces Hello Student.

14. Which function definition uses a default parameter ?

- A) def add(a, b):
- B) def add(a, b=10):
- C) def add(a=10, b):
- D) def add(a, b, c):

Answer: B) b=10 is a default parameter.

15. What is the output?

```
def calculate(a, b):
    sum_value = a + b
    product = a * b
    return sum_value, product
x, y = calculate(3, 4)
```

```
print(x, y)
```

- A) 7 12
- B) 12 7
- C) 7
- D) Error

Answer: A) calculate(3, 4) returns (7, 12), printed as 7 12.

16. Which of the following best represents modular design ?

- A) Writing the entire program inside one large function
- B) Dividing a program into functions such as read_data(), process_data(), and display_result()
- C) Avoiding functions
- D) Using only global variables

Answer: B) Separate functions for reading, processing, and displaying results are modular.

17. Consider:

```
def preprocess(data):
```

```
    data = data / 10
```

```
    return data
```

```
def calculate(data):
```

```
    return data * 5
```

```
x = preprocess(20)
```

```
y = calculate(x)
```

```
print(y)
```

What is the output?

- A) 2
- B) 10
- C) 20
- D) 100

Answer: B) preprocess(20) is 2, then calculate(2) is 10.

18. Which approach is most appropriate for writing a modular data -processing program?

- A) One large block containing all operations
- B) Separate functions for input, preprocessing, analysis, and output
- C) Only global variables
- D) Repeating the same code multiple times

Answer: B) Use separate functions for input, preprocessing, analysis, and output.

19. What is the output?

```
def modify(x):
```

```
    x = x + 10
```

```
    a = 5
```

```
    modify(a)
```

```
print(a)
```

- A) 5
- B) 10
- C) 15
- D) Error

Answer: A) Integers are immutable; rebinding local x does not change a.

20. Which of the following improves function reusability ?

- A) Hard -coding values inside the function
- B) Using appropriate parameters
- C) Using many global variables
- D) Duplicating the function

Code -Writing / Practical Questions

Answer: B) Parameters make functions reusable for different inputs.

21. Write a Python function called `calculate_average()` that accepts a list of numbers and returns

their average. Do not use NumPy.

Answer: `calculate_average(numbers)` returns `sum(numbers)/len(numbers)`; raise `ValueError` for an empty list.

```
def calculate_average(numbers):
    if not numbers:
        raise ValueError("numbers must not be empty")
    return sum(numbers) / len(numbers)

print(calculate_average([2, 4, 6, 8]))
```

22. Write a Python function called `find_maximum()` that accepts a list of numbers and returns the largest value without using Python's built-in `max()` function .

Answer: `find_maximum(numbers)` scans values without calling `max()`; raise `ValueError` for an empty list.

```
def find_maximum(numbers):
    if not numbers:
        raise ValueError("numbers must not be empty")
    largest = numbers[0]
    for number in numbers[1:]:
        if number > largest:
            largest = number
    return largest

print(find_maximum([3, 9, 2, 7]))
```

23. Write a modular Python program that contains separate functions to:

- Read a list of numbers
- Calculate the mean
- Calculate the maximum
- Display the results

Answer: Separate input, mean, maximum, and display into functions. The example reads a space-separated list.

```
def read_numbers():
    return [float(value) for value in input("Enter numbers separated by spaces: ").split()]

def calculate_mean(numbers):
    if not numbers:
        raise ValueError("numbers must not be empty")
    return sum(numbers) / len(numbers)
```

```

def calculate_maximum(numbers):
    if not numbers:
        raise ValueError("numbers must not be empty")
    largest = numbers[0]
    for number in numbers[1:]:
        if number > largest:
            largest = number
    return largest

def display_results(mean, maximum):
    print("Mean:", mean)
    print("Maximum:", maximum)

def main():
    numbers = read_numbers()
    display_results(calculate_mean(numbers), calculate_maximum(numbers))

if __name__ == "__main__":
    main()

```

24. Write a Python function called `count_even()` that accepts a list of integers and returns the number of even numbers in the list.

Answer: `count_even(numbers)` counts integers divisible by 2.

```

def count_even(numbers):
    return sum(1 for number in numbers if number % 2 == 0)

print(count_even([1, 2, 4, 7, 8]))

```

25. Write a modular Python program to calculate the accuracy of a classification model using two functions:

- `calculate_accuracy(y_true, y_pred)`
- `display_result(accuracy)`

Do not use Scikit -learn's built -in accuracy function.

Answer: Implement accuracy as the number of equal predictions divided by the number of samples, and display the result. Validate lengths and empty input.

```

def calculate_accuracy(y_true, y_pred):
    if len(y_true) != len(y_pred):
        raise ValueError("y_true and y_pred must have equal lengths")
    if not y_true:
        raise ValueError("inputs must not be empty")
    correct = sum(actual == predicted for actual, predicted in zip(y_true, y_pred))
    return correct / len(y_true)

def display_result(accuracy):
    print(f"Accuracy: {accuracy:.2%}")

y_true = [1, 0, 1, 1]
y_pred = [1, 0, 0, 1]
display_result(calculate_accuracy(y_true, y_pred))

```

26. Write a Python function that accepts a list of numbers and returns two values: mean and standard deviation .

Answer: Return the arithmetic mean and population standard deviation (divide squared deviations by n).

```

import math

def mean_and_standard_deviation(numbers):
    if not numbers:
        raise ValueError("numbers must not be empty")
    mean = sum(numbers) / len(numbers)
    variance = sum((value - mean) ** 2 for value in numbers) / len(numbers)
    return mean, math.sqrt(variance)

print(mean_and_standard_deviation([2, 4, 6, 8]))

```

27. Write a function called `normalize()` that accepts a list of numerical values and performs min -

max normalization:

$x_{\text{normalized}} = \frac{x - x_{\text{min}}}{x_{\text{max}} - x_{\text{min}}}$

$x_{\text{normalized}} = \frac{x - x_{\text{min}}}{x_{\text{max}} - x_{\text{min}}}$

Do not use Scikit -learn's preprocessing functions.

Answer: Apply min-max normalization; if all inputs are equal, return zeros because the denominator is zero.

```
def normalize(values):
    if not values:
        raise ValueError("values must not be empty")
    minimum = min(values)
    maximum = max(values)
    if maximum == minimum:
        return [0.0 for _ in values]
    return [(value - minimum) / (maximum - minimum) for value in values]

print(normalize([10, 20, 30, 40]))
```

28. Write a modular Python program for a simple machine -learning preprocessing pipeline

using separate functions:

`load_data()`

↓

`remove_missing_values()`

↓

`normalize_data()`

↓

`display_data()`

Answer: Create load, missing-row removal, per-column min-max normalization, and display functions. The example uses in-memory records; `load_data` can be replaced with a CSV reader.

```
def load_data():
    return [
        {"temperature": 20.0, "pressure": 100.0},
        {"temperature": 25.0, "pressure": 110.0},
        {"temperature": None, "pressure": 105.0},
        {"temperature": 30.0, "pressure": 120.0},
    ]

def remove_missing_values(rows):
    return [row for row in rows if all(value is not None for value in row.values())]

def normalize_data(rows):
    if not rows:
        return []
    columns = rows[0].keys()
    minima = {column: min(row[column] for row in rows) for column in columns}
    maxima = {column: max(row[column] for row in rows) for column in columns}
    normalized = []
    for row in rows:
        result = {}
        for column in columns:
            low, high = minima[column], maxima[column]
            result[column] = 0.0 if high == low else (row[column] - low) / (high - low)
        normalized.append(result)
    return normalized

def display_data(rows):
    for row in rows:
        print(row)

def main():
```

```
data = remove_missing_values(load_data())
display_data(normalize_data(data))

if __name__ == "__main__":
    main()
```

29. Write a Python function to calculate the Euclidean distance between two points without

using any built-in distance function.

Answer: Compute the square root of the sum of squared coordinate differences, after checking equal dimensions.

```
import math

def euclidean_distance(point_a, point_b):
    if len(point_a) != len(point_b):
        raise ValueError("points must have the same number of coordinates")
    return math.sqrt(sum((a - b) ** 2 for a, b in zip(point_a, point_b)))

print(euclidean_distance([1, 2], [4, 6]))
```